

基于大模型编码智能体的自进化 eBPF 网络入侵防御系统

硕士学位论文开题报告

研究生：李兆曦

指导教师：徐恪教授

清华大学计算机科学与技术系

2026 年 6 月



目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

TCP/IP 跨层歧义持续催生 off-path 协议攻击

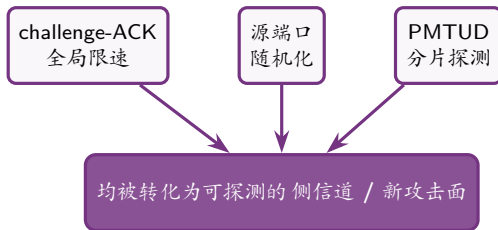
- TCP/IP 协议栈的跨层交互存在语义歧义，长期催生 **off-path**（链外）协议攻击——攻击者无需位于通信路径上，即可推断或注入连接状态：
 - challenge-ACK 侧信道¹（CVE-2016-5696，全局限速可被探测）
 - 混合 **IPID** 分配²、PMTUD 分片机制³、NAT / Wi-Fi 链路劫持⁴...
- 近期 **CCS'25** 工作⁵：利用 **PMTUD** 打破“IP 地址共享”场景下的 TCP 连接隔离
 - 单次攻击平均约 **220 s**，成功率约 **70%**
 - 实测 **50** 个真实网络中 **38** 个存在脆弱性

核心判断

协议攻击面并非偶发，而是植根于设计层面的跨层歧义，会被持续不断地重新发掘⁶。

规律：防御机制本身在制造新的攻击面

- 大量新攻击面恰由防御机制自身引入——“补丁”反而成为下一处侧信道：



启示

攻击与防御同源且相互转化 $\Rightarrow$ 防御须持续演化，一次性静态修补难以为继。

现有防御的三大结构性缺陷

1 零散

各攻击面分别打补丁，缺乏统一框架，防御策略碎片化、难以协同。

2 滞后

先有攻击曝光、再被动响应，从披露到修复存在显著时间窗口。

3 依赖人工编写内核代码

- 内核协议栈改动周期长：评审、回归、上线缓慢
- 易出错：一处缺陷即可断网或误杀正常流量
- 知识难以沉淀：经验留存于个人，复用困难

症结：防御的生产方式仍是人工、低速、难以累积的。

两项技术趋于成熟，使自动化防御成为可能

eBPF 内核可编程

- 热加载：无需修改内核源码、无需重启
- 贴近协议栈：XDP / TC 钩子在数据面线速处理
- 内置 **verifier**：对程序施加形式化安全验证（有界、内存安全、可终止）⁷

大模型编码智能体

- 能够编写、修改、调试系统级代码
- 具备工具调用、反思与迭代能力
- 已能在反馈闭环中自主推进复杂任务⁸⁻⁹
- 代表性产品 / 框架：Claude Code、Cursor、Codex、Devin、SWE-agent¹⁰⁻¹¹

契机

二者结合:eBPF 作可热插拔、可验证的内核执行底座, 智能体作自动化代码生产——使“由智能体持续维护内核防御”可行。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

核心研究问题

研究问题

能否让大模型编码智能体自主维护一套运行于内核的 **eBPF** 防御库，
使其持续进化，以应对不断涌现的协议攻击？

- 目标不是“一次性合成单个防御程序”，而是构建一个长期自主运转的防御系统；
- 核心在于自主性（探索—合成—验证—部署的闭环）与进化性（防御能力单调累积、不后退）。

四项关键挑战

1 可靠性

智能体编写的内核代码若含缺陷，可能直接断网或误杀正常流量——必须确保“先验证安全、后部署”。

2 探索高效性

协议状态空间巨大且充满噪声，如何高效发现真正有价值的攻击面，而非无效扰动？

3 非梯度演化的收敛性

防御以代码为载体演化，如何保证其单调进化、不腐化、不回退（避免灾难性遗忘）？

4 评测

如何量化系统对未知攻击的自主防御能力，给出可复现、可对比的度量？

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

相关工作（一）：协议攻防与 eBPF 运行时防御

协议攻击与检测

- 攻击：off-path 协议攻击系列工作（challenge-ACK、IPID、PMTUD 等）⁵⁻⁶
- 检测：SCENT¹²、SCAD¹³ 等侧信道检测方法
- 现状：防御零散、滞后，缺少统一且持续演化的框架

eBPF 运行时防御

- 系统：Katran¹⁴、Cilium¹⁵、Falco¹⁶、Tetragon¹⁷——能力强，但策略均由人工编写
- 验证：PREVAIL⁷、K2¹⁸ 等围绕 verifier 的程序分析与优化

相关工作（二）：大模型用于安全与方法论基础

大模型 × 安全

- 进攻：PentestGPT¹⁹、Big Sleep²⁰
- 合成 eBPF：Kgent / KEN²¹——由自然语言合成 eBPF，是最直接的对比基线
 - 局限：一次性合成，非自主、持续维护
- 工程载体：Claude Code / Cursor / Codex / Devin / SWE-agent^{10-11,22}——通用而非领域专用，需专门 harness

方法论基础

- 非梯度演化范式²³
- 程序演化搜索：FunSearch⁸、AlphaEvolve⁹
- 自演化智能体：Darwin-Gödel Machine²⁴
- 探索机制：ICM²⁵、RND²⁶（基于预测误差与新颖性）
- 多样化解集：Novelty Search²⁷、MAP-Elites²⁸

上述方法论共同支撑“以可验证代码为载体的非梯度演化”。

定位对比：本文与代表性工作

对比对象	已有范式	本文范式
Kgent / KEN ²¹ Tetragon / Falco ¹⁶⁻¹⁷ 小模型（权重学习）	一次性合成 eBPF 人工编写策略 知识沉淀为权重	长期自主维护的防御库 自进化地生成与更新策略 知识沉淀为可验证代码

- 共同差异：由一次合成 / 人工 / 权重 → 持续、自主、可验证的代码演化。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

核心思路：防御演化 (Defensive Evolution)

什么是“防御演化”

让编码智能体持续维护并演化一套运行在内核的防御代码: 把防御能力沉淀为可验证的 **eBPF** 代码 (而不是模型权重), 随攻击的出现不断增补、修正。

以可验证代码为载体的优势:

- 容量无界 (不受参数量限制)
- 抗灾难性遗忘 (旧防御不被覆盖)
- 可解释 (每条防御对应一段代码)
- 线速执行 (XDP 数据面)
- 可形式化验证 (verifier 背书)

四项研究内容

① 范式形式化 + 系统框架 + 专用 harness

界定“以可验证代码为载体的演化”；把 eBPF 工具、verifier 反馈、沙箱、靶场、档案封装成智能体的工具接口。

② 攻击面探索

以新颖性与覆盖度为信号，发现现有防御覆盖不到的攻击。

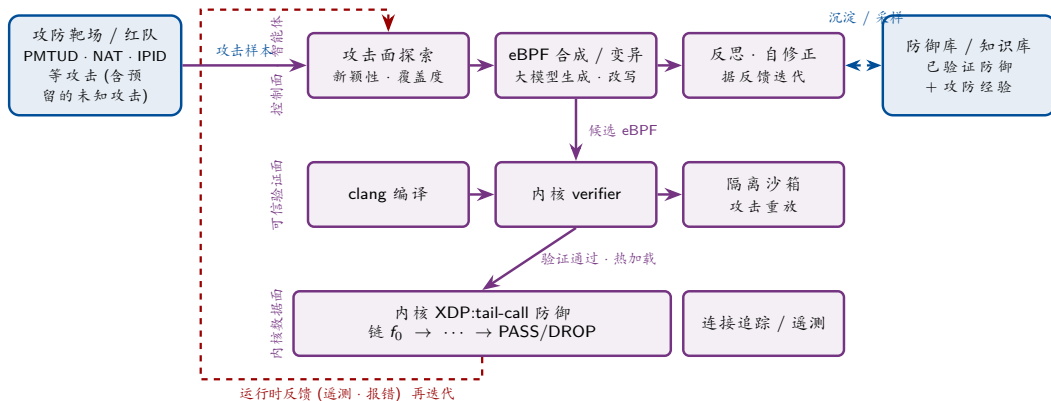
③ 进化式 eBPF 合成 + 可信加载闭环

大模型生成 / 改写 eBPF，经多层验证后热加载到内核。

④ 防御库维护 + 攻防协同演化评测

沉淀多样化防御，量化对已知 / 未知攻击的防御效果。

系统分层架构



控制面 (智能体) 生成候选 可信验证面把关 内核数据面执行; 反馈回灌驱动持续迭代, 有效防御沉淀入知识库。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

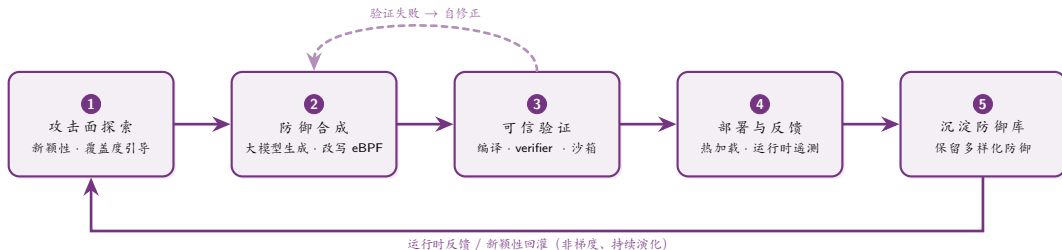
⑤ 技术路线

⑥ 创新点

⑦ 工作计划

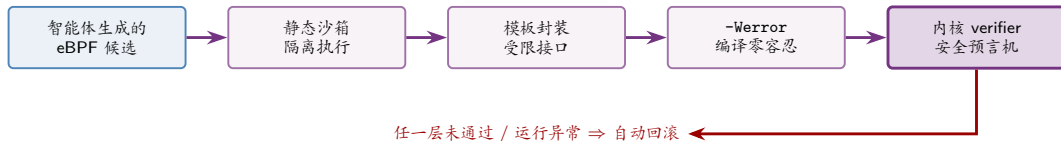
⑧ 预期成果

技术路线：防御演化闭环（五阶段）



每轮迭代将新发现的攻击面转化为一段经 verifier 背书的 eBPF 防御，并沉淀入防御库。

可信保障：四层安全约束 + 自动回滚



思路：现成安全设施 + 智能体闭环

这四层（沙箱、模板、-Werror、内核 verifier）都是现成的基础设施；本文不重造它们，而是把它们接入智能体闭环——以编译 / verifier 报错作反馈驱动自修正，沙箱重放把关，异常即自动回滚，从而可靠地实现内核代码自动化。

工程基线：已有开源系统积累

PacketScope / Guarder

- **XDP** 数据面：在网卡驱动层线速处理报文
- **bpf2go** 一键编译：Go 侧封装，eBPF 程序构建与加载流程化
- 规则热加载：策略可在运行时动态更新，无需重启
- 已具备 **Agent** 可编程 **eBPF** 原型，配套 **312** 项测试

已具备数据面、编译链与热加载基础设施，闭环可在现有底座上快速搭建。

评测设计：攻防协同演化

评测方式

红队与防御系统协同演化; 攻击集分为训练种子与预留的未知攻击, 考察无人工补丁介入下的自主防御。

度量指标

- 已知攻击: 检出率 / 误报率
- 未知攻击 (预留测试): 随迭代的防御覆盖提升
- 性能: XDP 吞吐损失 / 时延
- 自动化程度 (人工介入量)

对比与消融

- Kgent / KEN(一次性合成)
- Tetragon / Falco(人工策略)
- 消融: 去探索 / 去档案 / 去自修正

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

创新点

1 范式

让编码智能体持续维护与演化内核防御代码——防御沉淀为“可验证代码”而非模型权重(即防御演化)。

2 专用 harness

为内核 eBPF 防御构建领域专用的智能体 **harness**(领域工具 + 反馈闭环), 而非直接套用 Claude Code / Cursor 等通用编码智能体。

3 攻防协同演化评测基准

构建含预留的未知攻击的评测, 量化对已知 / 未知攻击的防御效果、误报与开销。

目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

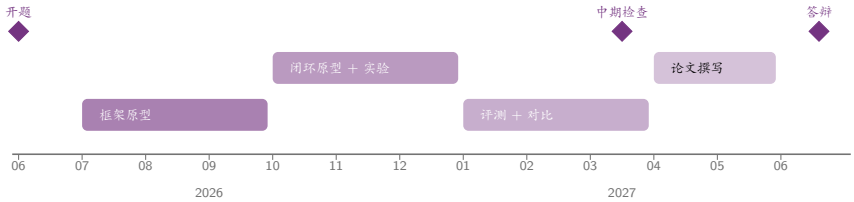
⑥ 创新点

⑦ 工作计划

⑧ 预期成果

工作计划 (2026.06 – 2027.06)

时间	工作内容
2026.06	开题报告
2026.07 – 09	防御演化框架原型：形式化定义 + 系统骨架
2026.10 – 12	闭环原型 + 初步实验（探索 / 合成 / 验证 / 部署 / 档案）
2027.01 – 03	评测 + 对比实验 + 中期检查
2027.04 – 05	论文撰写、投稿 + 预答辩 + 送审
2027.06	学位论文答辩



目录

① 研究背景与动机

② 研究问题与挑战

③ 相关工作

④ 研究内容与方法

⑤ 技术路线

⑥ 创新点

⑦ 工作计划

⑧ 预期成果

预期成果

- 学位论文：完成硕士学位论文 1 篇。
- 开源系统：开源自进化 **eBPF** 防御系统、防御演化专用 **harness** 及配套攻防评测基准。
- 社区贡献：对发现的协议 / 实现缺陷，向 **IETF / Linux** 进行负责任披露。

成果定位

既贡献新范式与系统，也贡献可复现的评测基准，推动“智能体自主维护内核安全”方向的后续研究。

参考文献 I

- [1] CAO Y, WANG Z, QIAN Z, et al. Off-Path TCP Exploits: Global Rate Limit Considered Dangerous[C]//25th USENIX Security Symposium. 2016: 209-225.
- [2] FENG X, FU C, LI Q, et al. Off-Path TCP Exploits of the Mixed IPID Assignment[C]//Proc. 2020 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2020: 1323-1335. DOI: 10.1145/3372297.3417884.
- [3] FENG X, LI Q, SUN K, et al. PMTUD is not Panacea: Revisiting IP Fragmentation Attacks against TCP[C]//29th Network and Distributed System Security Symposium (NDSS). 2022. DOI: 10.14722/ndss.2022.24381.
- [4] YANG Y, FENG X, LI Q, et al. Exploiting Sequence Number Leakage: TCP Hijacking in NAT-Enabled Wi-Fi Networks[C]//31st Network and Distributed System Security Symposium (NDSS). 2024.
- [5] FENG X, LI Z, LI Q, et al. Off-Path TCP Exploits: PMTUD Breaks TCP Connection Isolation in IP Address Sharing Scenarios[C]//Proc. 2025 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2025. DOI: 10.1145/3719027.3744888.
- [6] FENG X, LI Q, SUN K, et al. Exploiting Cross-Layer Vulnerabilities: Off-Path Attacks on the TCP/IP Protocol Suite[J]. Communications of the ACM, 2025, 68(3): 48-59. DOI: 10.1145/3689819.
- [7] GERSHUNI E, AMIT N, GURFINKEL A, et al. Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions[C]//Proc. 40th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI). 2019: 1069-1084. DOI: 10.1145/3314221.3314590.
- [8] ROMERA-PAREDES B, BAREKATAIN M, NOVIKOV A, et al. Mathematical Discoveries from Program Search with Large Language Models[J]. Nature, 2024, 625: 468-475. DOI: 10.1038/s41586-023-06924-6.

参考文献 II

- [9] NOVIKOV A, V N, EISENBERGER M, et al. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery[J]. ArXiv preprint arXiv:2506.13131, 2025.
- [10] Anthropic. Claude Code: Agentic Coding in the Terminal[Z]. <https://www.anthropic.com/claude-code>. 2025.
- [11] YANG J, JIMENEZ C E, WETTIG A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[C]//Advances in Neural Information Processing Systems (NeurIPS). 2024.
- [12] CAO Y, WANG Z, QIAN Z, et al. Principled Unearthing of TCP Side Channel Vulnerabilities[C]//Proc. 2019 ACM SIGSAC Conf. on Computer and Communications Security (CCS). 2019: 211-224. DOI: 10.1145/3319535.3354250.
- [13] MAN K, WANG Z, HAO Y, et al. SCAD: Towards a Universal and Automated Network Side-Channel Vulnerability Detection[C]//33rd USENIX Security Symposium. 2024.
- [14] SHIROKOV N, DASINENI R. Open-sourcing Katran, a Scalable Network Load Balancer[Z]. Meta Engineering Blog. 2018.
- [15] GRAF T. EBPF – The Future of Networking and Security[Z]. Cilium / Isovalent Blog. 2020.
- [16] The Falco Authors. Falco: Cloud Native Runtime Security[Z]. CNCF Project. 2018.
- [17] Isovalent. Tetragon: eBPF-based Security Observability and Runtime Enforcement[Z]. Isovalent / CNCF. 2022.
- [18] XU Q, WONG M D, WAGLE T, et al. Synthesizing Safe and Efficient Kernel Extensions for Packet Processing[C]//Proc. 2021 ACM SIGCOMM Conference. 2021. DOI: 10.1145/3452296.3472929.
- [19] DENG G, LIU Y, MAYORAL-VILCHES V, et al. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing[C]//33rd USENIX Security Symposium. 2024: 847-864.

参考文献 III

- [20] GLAZUNOV S, BRAND M. Project Naptime: Evaluating Offensive Security Capabilities of Large Language Models[Z]. Google Project Zero Blog. 2024.
- [21] YANG Y, ZHENG Y, CHEN M, et al. Kgent: Kernel Extensions Large Language Model Agent[C]//Proc. ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions. 2024: 30-36. DOI: 10.1145/3672197.3673434.
- [22] Anysphere. Cursor: The AI Code Editor[Z]. <https://www.cursor.com>. 2024.
- [23] WENG J. Learning Beyond Gradients[Z]. <https://github.com/Trinkle23897/learning-beyond-gradients>. 2026.
- [24] ZHANG J Z, HU S, LU C, et al. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents[J]. ArXiv preprint arXiv:2505.22954, 2025.
- [25] PATHAK D, AGRAWAL P, EFROS A A, et al. Curiosity-driven Exploration by Self-supervised Prediction[C]//Int. Conf. on Machine Learning (ICML). 2017.
- [26] BURDA Y, EDWARDS H, STORKEY A, et al. Exploration by Random Network Distillation[C]//International Conference on Learning Representations (ICLR). 2019.
- [27] LEHMAN J, STANLEY K O. Abandoning Objectives: Evolution Through the Search for Novelty Alone[J]. Evolutionary Computation, 2011, 19(2): 189-223. DOI: 10.1162/EVCO_a_00025.
- [28] MOURET J B, CLUNE J. Illuminating Search Spaces by Mapping Elites[J]. ArXiv preprint arXiv:1504.04909, 2015.

敬请各位老师批评指正

谢 谢!